

SQL INDEXY

Ing. Ladislav Körösi, PhD.
ÚRK, FEI STU

ladislav.korosi@stuba.sk

Literatúra (Zdroj):

<https://www.vertabelo.com/blog/technical-articles/all-about-indexes-part-2-mysql-index-structure-and-performance>

Database indexes speed up data retrieval operations. But there is a price we pay for these benefits.

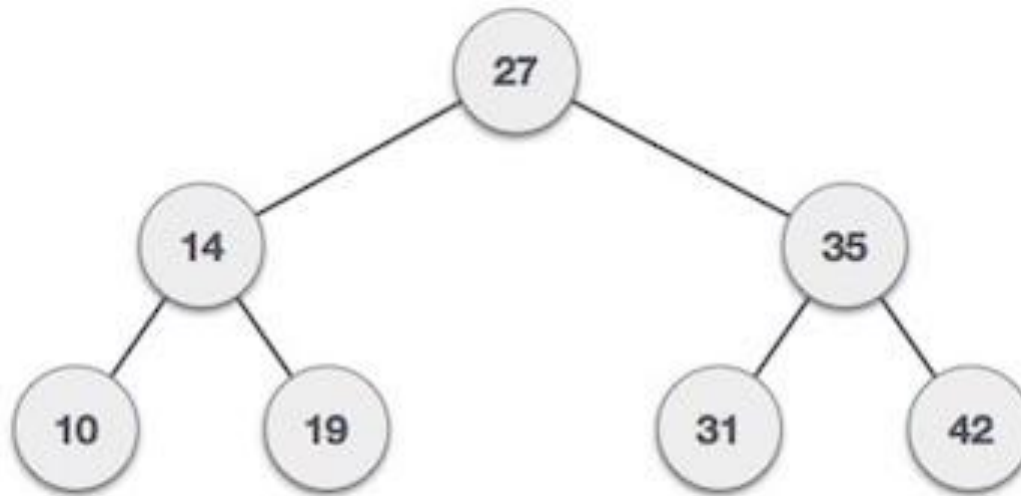
MySQL supports a few different index types. The most important are **BTREE** and **HASH**. These types are also the most common types in other DBMSs.

Common node types

- **Root node** – the topmost node in the tree structure
- **Child nodes** – nodes that another node (the parent node) points to
- **Parent nodes** – nodes that point to other nodes (child nodes)
- **Leaf nodes** – nodes that have no child nodes (located on the bottom of the tree structure)
- **Internal nodes** – all “non-leaf” nodes, including the root node
- **External nodes** – another name for leaf nodes

Binary Search Trees (BST)

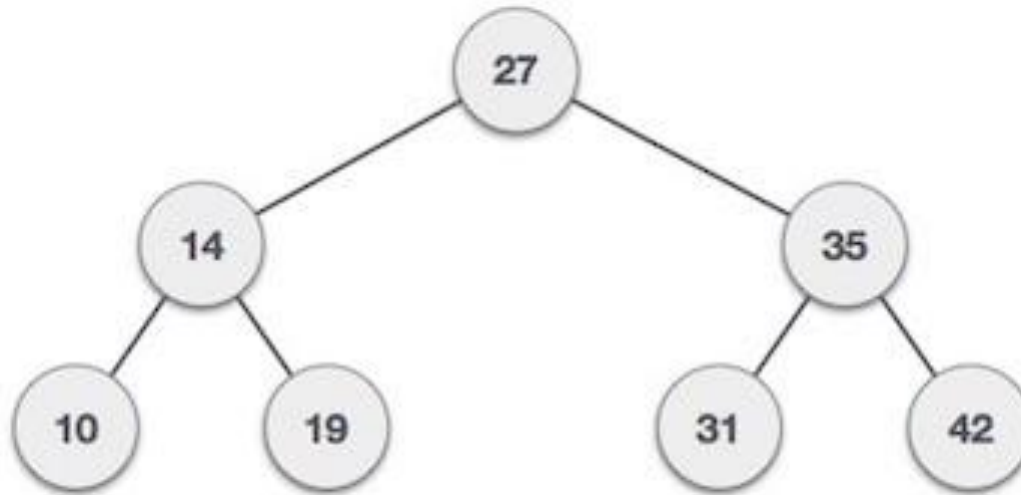
These structures are not database indexes. A BST is a binary tree in which the nodes are sorted. In the picture above, we see that the values on the left of node X are smaller than X, and the values on the right of node X are greater. This is how any BST is organized.



Binary Search Trees (BST)

Search for a value in the BST.

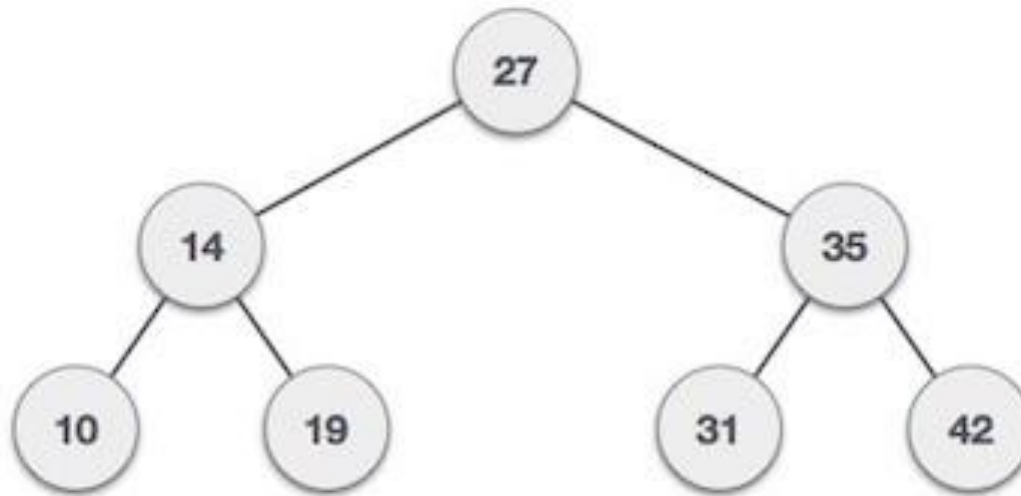
Adding a new value to the BST starts just like searching for a value (add „30“).



Binary Search Trees (BTS)

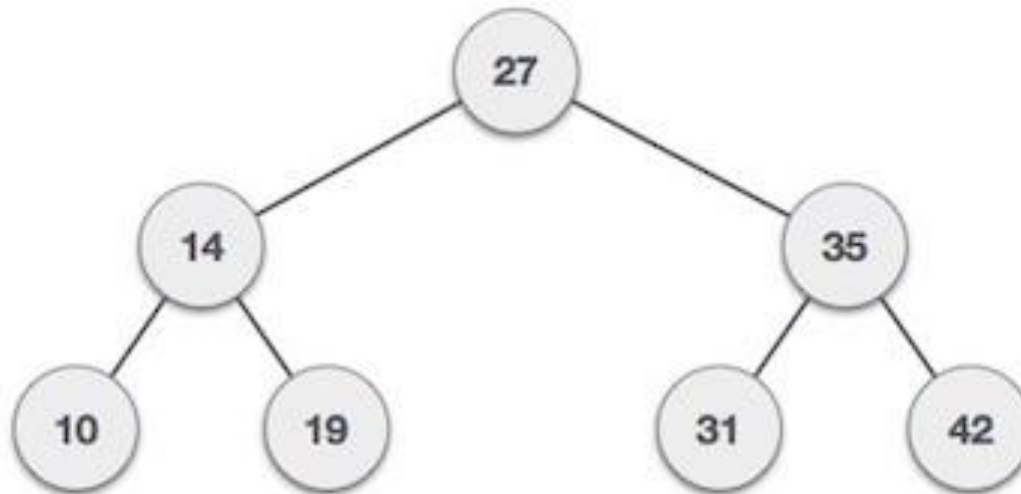
Deleting a node that has no child nodes (“10”, “19”, “31” and “42”)

Deleting a node that has one child node - We'll replace the deleted node with the child node (delete „30“)



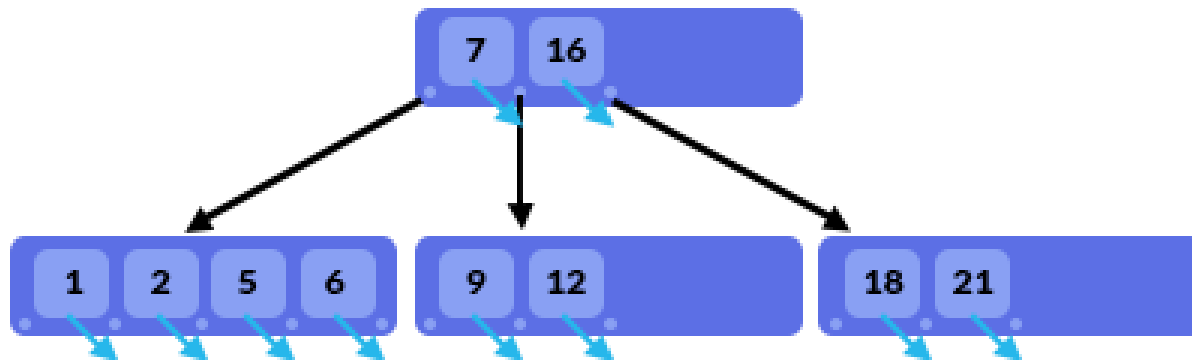
Binary Search Trees (BTS)

Deleting a node that has two child nodes - This is the most complex case. In this situation, we'll find the smallest element in the right subtree of the deleted node and put it in place of the deleted node (delete „27“).



B-Trees

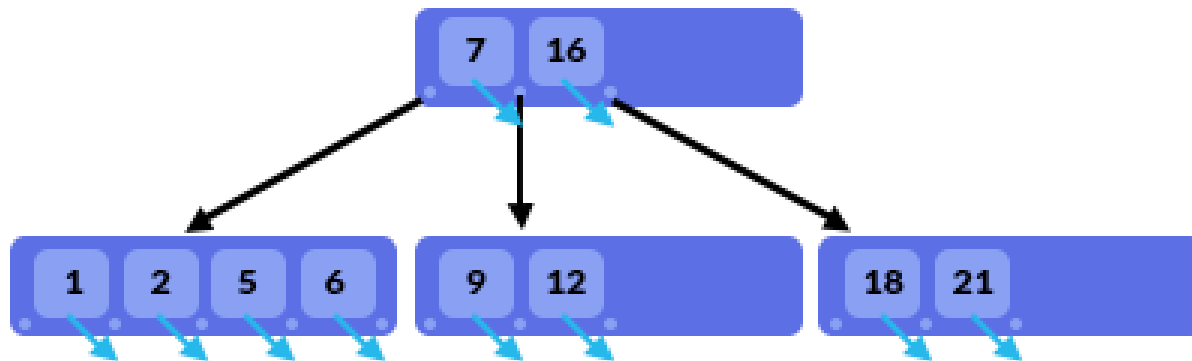
The **B-Tree** is the basic index structure for most MySQL storage engines. Each node in a B-Tree has between d and $2d$ values. Values in each node are sorted. Each node has between 0 to $2d+1$ child nodes. Each child node is attached before, after, or between values. (In the above graphic, values “9” and “12” come between values “7” and “16”). The values in B-Tree are sorted in a similar way to those in a binary search tree.



B-Trees

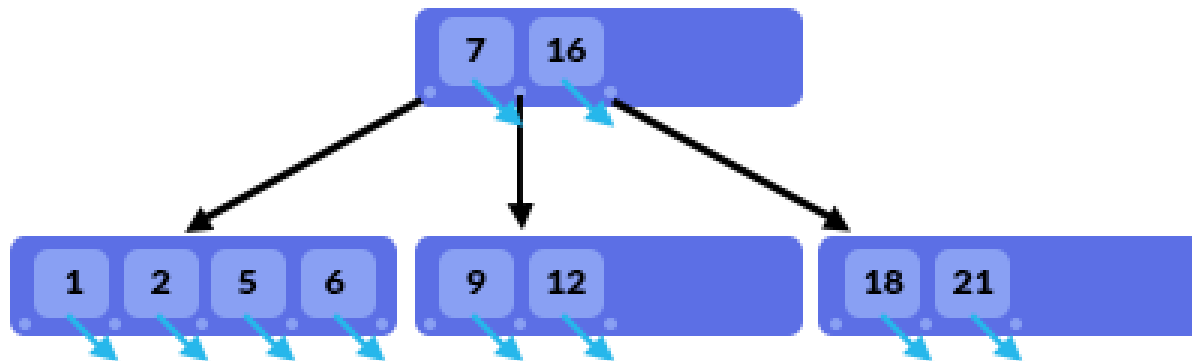
In contrast to a BST, a B-Tree is a balanced tree: all branches of the tree have the same length.

Searching for values in a B-Tree also corresponds to searching in a BST. First we check if the value is present in the root node. If it isn't, we select the appropriate child node, and look for the value in that node.



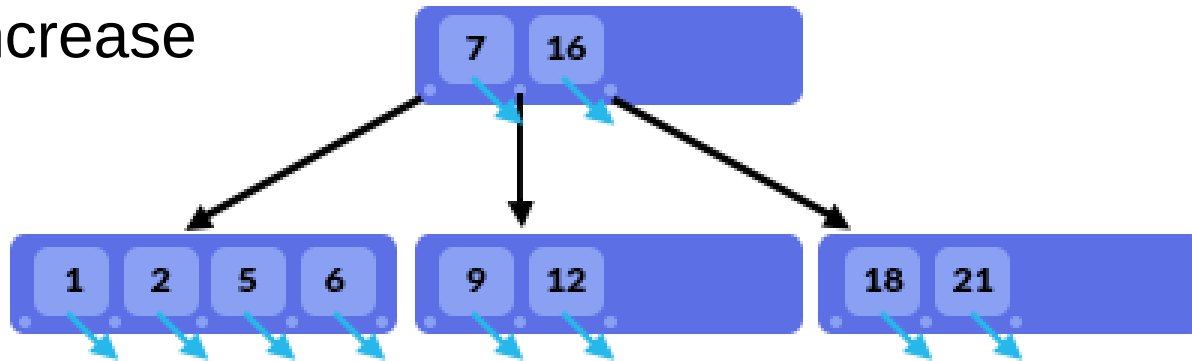
B-Trees

Adding a value has to maintain both the order of value and the balance of the tree. First we'll find the leaf node where the values should be added. If there is enough space in the leaf node, we'll simply add the value; the structure and the tree depth won't change (add „22“ – easy, add „4“ - overflow).



B-Trees

If we want to add the value “4”, we would split the first leaf node. This always happens in a case of overflow. Values “1” and “2” would form a new leaf node (the left one); value “4” would be the middle node (and inserted into the parent node), while values “5” and “6” would form another new leaf node (the right one). We would split the parent node in the same manner if an overflow happens there. In extreme cases, we’d have to split the root node and the tree depth would increase



B-Trees

When we want to **delete a value** from the B-Tree, we'll locate that value and remove it. If that deletion caused underflow (the number of values stored in a node is too low) we'll have to merge nodes together. This is exactly the opposite from what happens when we're adding values. We'll merge that node with the neighboring node that contains the most values. If the newly merged node contains too many values (overflow) we'll split it to the left, middle, and right and rearrange the tree structure.

B-Trees

When B-Tree is used in database index, each node holds the index value and the **pointer** to the row it comes from. For example, if the index is on the column *id*, then the tree will hold both the value "7" and a pointer to the row where *id*=7. When we search for the row where *id*=7, we can quickly find the value 7 in the B-Tree and then follow the pointer to get the actual row.

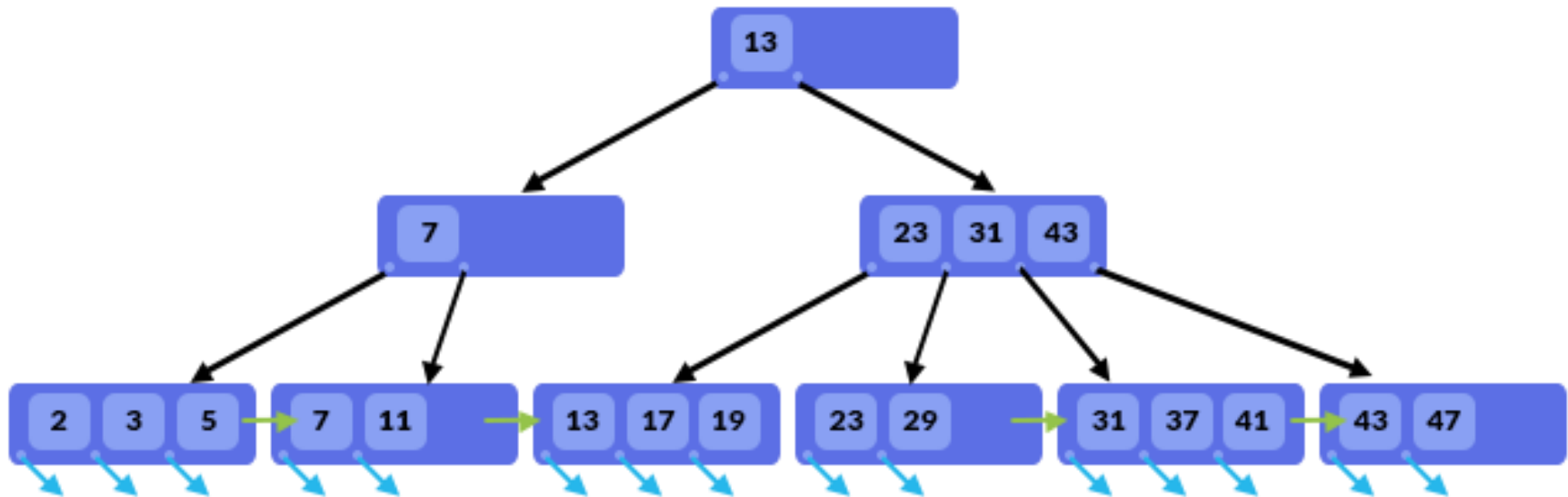
The important thing to remember is that **in the B-tree structure each node contains key values, pointers to the values**, and pointers to child nodes with smaller and larger values than the value in the parent node. Of course, leaf nodes don't point to any child nodes.

B-Trees

With a B-Tree index, we can use equality operators (= or <=>), operators that will result in a range (>, <, >=, <=, BETWEEN), and the LIKE operator.

B+Trees

The **B+Tree** structure is similar to the B-Tree structure. The InnoDB storage engine uses a B+Tree structure to store indexes.



B+Trees

The most important differences are:

- The internal nodes only store values; they don't store pointers to actual rows. The leaf nodes store the values and row pointers. This reduces the internal nodes' size, allowing the storage of more nodes on the same memory page. In turn, this increases the branching factor. As the branching factor grows, the height of the tree decreases and that results in fewer disk I/O operations.

B+Trees

The most important differences are:

- Leaf nodes in the B+Tree are linked, so we can conduct a full scan with only one pass. That is very helpful when we need to find all the data in a given range. Because row pointers are stored in both internal and leaf nodes, this isn't possible in a B-Tree.

B+Trees

The most important differences are:

- Performing delete operations in a B+tree structure is much easier than in a B-tree. This is because we don't need to remove values from the internal nodes. In a B+Tree structure, we'll have the same value repeated; in a B-Tree structure, each value is unique. In B+Tree, we'll store one value and one data pointer in the leaf node, but that value could also be stored in internal nodes (where it's used to point to child nodes).

B+Trees

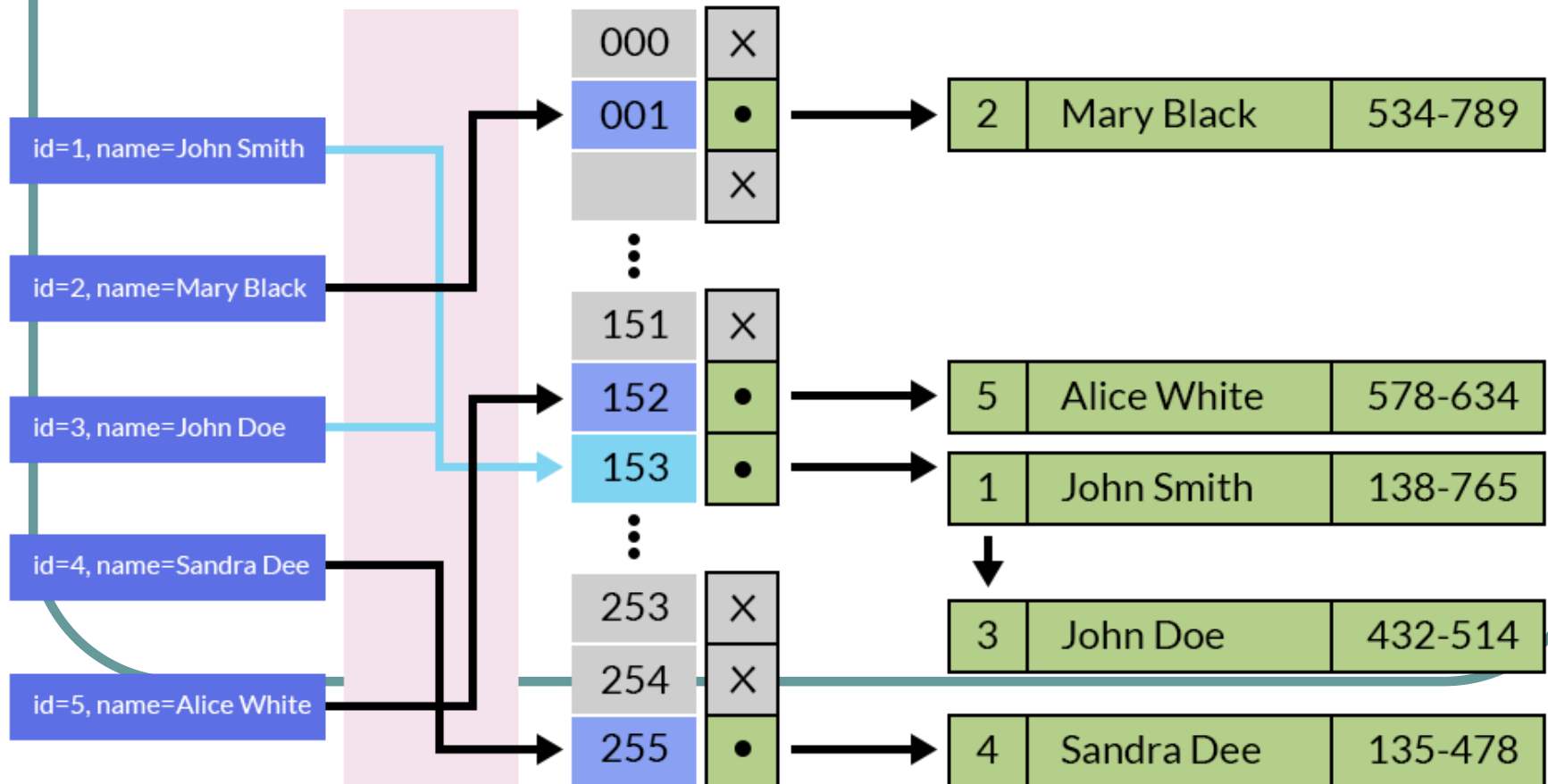
The most important differences are:

- The advantage of B-Tree is that we can find values that lie close to the root fairly quickly, while in B+Tree we would need to look all the way down to the leaf nodes for any value.

Hash Indexes

Hash indexes are directly related to the hashing technique.

**Hash
function Buckets**



Hash Indexes

On the left side, we see the set of key values used to find data. In this case, they are numeric values. The hash function is used to calculate the address where the actual data bucket is stored. That will give us the location of records related to each key value.

Values are stored in buckets based on the value of a hash function. When we want to **search** for a value, we'll use the hash function to calculate the address where our data could be stored. We'll look for the data in the bucket. If we find it, we're done. If we don't find our value, it means it's not in the index.

Hash Indexes

Adding new values works similarly: we'll use the hash function to calculate the address where we'll store our data. If that address is already occupied, we'll add new buckets and re-compute the hash function. Once again, we'll use the whole key as an input for our function. The result is the actual address (in disk memory) where we can find the desired data.

Updating or deleting values consists of first searching for a value and then applying the desired operation on that memory address.

Hash Indexes

Hash table indexes are very fast when testing for equality (= or <=>). This is because we're using the whole key and not just its parts. Individual parts can't help us when we want to find range, use the < or > operators, or speed up the ORDER BY part of the clause.

Engines

MySQL uses both BTREE (B-Tree and B+Tree) and HASH indexes. MyISAM use only BTREE indexes while MEMORY/HEAP and NDB can use both HASH and BTREE. MEMORY/HEAP and NDB will use the HASH index structure by default.